

Módulo 4 — Desarrollo de Aplicaciones con IA

TributIA

Evaluación Semana 4 — Despliegue, contenedores e infraestructura inicial

Equipo	11
Integrantes	Daniel Enrique Zaldaña Castillo · Edwin Adony Ulloa Díaz · Anthony Enrique Alvarenga Mayen
Docente	Ing. Marco Arévalo Zambrano
Fecha	1 de agosto de 2026
Repositorio	https://github.com/DanCastSV/TributIA

1. Descripción breve del servicio preparado

TributIA es una plataforma web Django que analiza documentos tributarios (facturas, constancias, comprobantes en PDF/PNG/JPG) mediante un pipeline de OCR (Tesseract) + NLP (spaCy) + LLM (Google Gemini), y expone tanto una interfaz web como una API interna versionada (`/api/v1/`). Para esta entrega se preparó el servicio completo (app Django + dependencias de sistema para OCR) para ejecutarse de forma reproducible dentro de un contenedor Docker, fuera del entorno personal de desarrollo.

2. Ruta elegida

Contenedor Docker, mediante `Dockerfile` + `docker-compose.yml` en la raíz del repositorio, con dos servicios: **web** (Django + gunicorn) y **db** (PostgreSQL 16) — la base de datos del contenedor se migró de SQLite a PostgreSQL como parte de esta entrega (sección 6b).

Se encontró un bloqueo inicial de entorno ("Virtualization support not detected" en Docker Desktop) por falta de WSL2/Virtual Machine Platform en Windows — documentado con causa raíz y solución en la sección 9.1.

3. Dockerfile

```
FROM python:3.13-slim

# tesseract-ocr: motor de OCR usado por core/ocr_utils.py (el paquete de
# idioma español se toma del tessdata/ empaquetado en el repo, no del
# sistema). poppler-utils: requerido por pdf2image para convertir PDF a
# imagen. build-essential: por si alguna dependencia de spaCy no trae
# wheel prebuilt para esta versión de Python y necesita compilar.
RUN apt-get update && apt-get install -y --no-install-recommends \
    tesseract-ocr \
    poppler-utils \
    build-essential \
    && rm -rf /var/lib/apt/lists/*

ENV PYTHONDONTWRITEBYTECODE=1 \
    PYTHONUNBUFFERED=1

WORKDIR /app

COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt

COPY . .

EXPOSE 8000

# migrate + collectstatic corren al iniciar el contenedor (no en build)
# porque settings.py exige SECRET_KEY/GEMINI_API_KEY vía os.environ[...],
# que solo están disponibles en runtime a través de .env / env_file.
CMD ["sh", "-c", "python manage.py migrate --noinput && python manage.py collectstatic --noinput && gunicorn tributia_project.wsgi:application --bind 0.0.0.0:8000 --workers 3 --timeout 120"]
```

docker-compose.yml:

```
services:
  db:
    image: postgres:16-alpine
    environment:
      - POSTGRES_DB=${POSTGRES_DB}
      - POSTGRES_USER=${POSTGRES_USER}
      - POSTGRES_PASSWORD=${POSTGRES_PASSWORD}
    volumes:
      - postgres_data:/var/lib/postgresql/data
    healthcheck:
      test: ["CMD-SHELL", "pg_isready -U ${POSTGRES_USER}"]
      interval: 5s
      timeout: 5s
      retries: 5

  web:
    build: .
    ports:
      - "8000:8000"
    env_file:
      - .env
    environment:
      - DEBUG=False
      - ALLOWED_HOSTS=localhost,127.0.0.1
      - POSTGRES_HOST=db
    depends_on:
      db:
        condition: service_healthy
    volumes:
      - ./media:/app/media

volumes:
  postgres_data:
```

4. .dockerignore

```
.git
.github
.venv
venv
env
__pycache__
*.pyc
*.pyo
.env
db.sqlite3
media
staticfiles
docs
tests
.vscode
.idea
Thumbs.db
*.md
```

Excluye secretos reales (`.env` — se inyectan vía `env_file` en runtime, nunca se copian a la imagen), datos de desarrollo (`db.sqlite3` , `media/`) y archivos no necesarios en runtime (`docs/` , `tests/` , markdown), para mantener la imagen liviana y sin datos sensibles horneados.

5. Variables de entorno

`.env.example` (sin valores reales):

```
SECRET_KEY=change-me
DEBUG=True
ALLOWED_HOSTS=localhost,127.0.0.1
GEMINI_API_KEY=your-gemini-api-key
EMAIL_HOST_USER=your-email@gmail.com
EMAIL_HOST_PASSWORD=your-gmail-app-password
```

Variable	Descripción	Obligatoria	Nueva en Semana 4
SECRET_KEY	Clave secreta de Django	Sí	
DEBUG	<code>True / False</code> . En contenedor se recomienda <code>False</code>	Sí	
ALLOWED_HOSTS	Hosts separados por coma permitidos por Django	Sí	✓
GEMINI_API_KEY	API key de Google Gemini	Sí	
EMAIL_HOST_USER / EMAIL_HOST_PASSWORD	Credenciales SMTP de Gmail	Sí	
POSTGRES_DB / POSTGRES_USER / POSTGRES_PASSWORD	Credenciales de PostgreSQL (servicio db)	Solo en Docker	✓

`DEBUG` y `ALLOWED_HOSTS` se agregaron en `tributia_project/settings.py` como configurables por entorno (antes: `DEBUG = True` y `ALLOWED_HOSTS = []` hardcodedos) — requisito para correr el servicio fuera de `manage.py runserver` .

6. Evidencia de construcción y ejecución

Construcción de la imagen (`docker compose build`):

```
$ docker compose build
Image tributia-web Building
#6 [1/6] FROM docker.io/library/python:3.13-slim
#7 [2/6] RUN apt-get update && apt-get install -y --no-install-recommends tesseract-ocr
poppler-utils build-essential
#9 [4/6] COPY requirements.txt .
#10 [5/6] RUN pip install --no-cache-dir -r requirements.txt
#10 72.85 Successfully installed Django-6.0.3 ... gunicorn-23.0.0 ... whitenoise-6.8.2 ...
spacy-3.8.14 ... google-generativeai-0.8.6
#11 [6/6] COPY . .
#12 exporting to image
#12 naming to docker.io/library/tributia-web:latest done
Image tributia-web Built
```

Build exitoso, sin errores de compilación (las dependencias de spaCy — `blis` , `thinc` — trajeron wheels prebuilt para Python 3.13).

Ejecución (`docker compose up -d`) y arranque:

```
$ docker compose up -d
Network tributia_default Created
Container tributia-web-1 Created
Container tributia-web-1 Started

$ docker compose logs web
web-1 | Operations to perform:
web-1 |   Apply all migrations: admin, auth, contenttypes, core, sessions
web-1 | Running migrations:
web-1 |   No migrations to apply.
web-1 | 132 static files copied to '/app/staticfiles', 396 post-processed, 2 skipped due to
conflict.
web-1 | [2026-08-01 21:43:20 +0000] [36] [INFO] Starting gunicorn 23.0.0
web-1 | [2026-08-01 21:43:20 +0000] [36] [INFO] Listening at: http://0.0.0.0:8000 (36)
web-1 | [2026-08-01 21:43:20 +0000] [36] [INFO] Using worker: sync
web-1 | [2026-08-01 21:43:20 +0000] [37] [INFO] Booting worker with pid: 37
web-1 | [2026-08-01 21:43:20 +0000] [38] [INFO] Booting worker with pid: 38
web-1 | [2026-08-01 21:43:20 +0000] [39] [INFO] Booting worker with pid: 39

$ docker compose ps
NAME                IMAGE                COMMAND                STATUS                PORTS
tributia-web-1      tributia-web         "sh -c 'python manag..." Up                    0.0.0.0:8000-
→8000/tcp
```

El contenedor migró la base de datos, corrió `collectstatic` (whitenoise sirviendo estáticos, sin nginx) y levantó gunicorn con 3 workers, con `DEBUG=False` y `ALLOWED_HOSTS=localhost,127.0.0.1` — la misma configuración que tendría un despliegue real, no `manage.py runserver` .

6b. Migración a PostgreSQL

Se agregó el servicio `db` (`postgres:16-alpine`) a `docker-compose.yml`, con volumen nombrado `postgres_data` y `healthcheck` (`pg_isready`); `web` espera a que Postgres esté sano antes de arrancar (`depends_on: condition: service_healthy`). `settings.py` elige el motor según el entorno:

```
if os.environ.get('POSTGRES_HOST'):
    DATABASES = {
        'default': {
            'ENGINE': 'django.db.backends.postgresql',
            'NAME': os.environ['POSTGRES_DB'],
            'USER': os.environ['POSTGRES_USER'],
            'PASSWORD': os.environ['POSTGRES_PASSWORD'],
            'HOST': os.environ['POSTGRES_HOST'],
            'PORT': os.environ.get('POSTGRES_PORT', '5432'),
        }
    }
else:
    DATABASES = {
        'default': {
            'ENGINE': 'django.db.backends.sqlite3',
            'NAME': BASE_DIR / 'db.sqlite3',
        }
    }
```

SQLite se mantiene como fallback en desarrollo local sin Docker, para no romper el flujo de trabajo actual del equipo.

```
$ docker compose up -d --build
#10 Successfully installed ... psycopg2-binary-2.9.10 ...
Image tributia-web Built
Volume tributia_postgres_data Created
Container tributia-db-1 Started
Container tributia-db-1 Healthy
Container tributia-web-1 Started

$ docker compose logs web
web-1 | Applying auth.0012_alter_user_first_name_max_length... OK
web-1 | Applying core.0001_initial... OK
web-1 | ...
web-1 | Applying core.0013_eventocalendario... OK
web-1 | Applying sessions.0001_initial... OK
web-1 | 132 static files copied to '/app/staticfiles', 396 post-processed, 2 skipped due to
web-1 | conflict.
web-1 | [INFO] Starting gunicorn 23.0.0
```

Las 30+ migraciones existentes corrieron limpio contra la base Postgres nueva, sin ningún ajuste a `core/models.py` — el ORM de Django abstraigo la diferencia de motor.

7. Prueba del endpoint `/health`


```
$ curl -s -w "\nHTTP_STATUS:%{http_code}\n" http://localhost:8000/api/v1/health/
{"status": "ok", "checks": {"base_datos": "ok", "tesseract": "ok", "gemini_api_key":
"configurada"}}
HTTP_STATUS:200
```

Confirma, **desde dentro del contenedor**, que la BD funciona, que `tesseract` instalado vía `apt-get` se encuentra correctamente, y que `GEMINI_API_KEY` llegó inyectada desde `.env`.

```
$ curl -s -o /dev/null -w "%{http_code}\n" http://localhost:8000/
200
# HTML devuelto referencia /static/css/style.5eaec3cdde5d.css
# (hash de cache-busting de CompressedManifestStaticFilesStorage / whitenoise)
```

8. Prueba del endpoint principal

Endpoint principal de IA: `POST /api/v1/analizar-documento/` . Requiere sesión autenticada; se probó con el flujo completo: login por curl → subida de un documento real (factura PDF) → pipeline completo (OCR/texto embebido, spaCy, Gemini) corriendo dentro del contenedor.

```
# 1. Login (obtiene sessionid)
$ curl -s -c cookies.txt http://localhost:8000/login/ -o /dev/null
$ CSRF=$(grep csrftoken cookies.txt | awk '{print $7}')
$ curl -s -b cookies.txt -c cookies.txt -X POST \
  -d "username=demo_docker&password=DemoDocker123!&csrfmiddlewaretoken=$CSRF" \
  -e "http://localhost:8000/login/" \
  -w "HTTP_STATUS_LOGIN:%{http_code}\n" -o /dev/null \
  http://localhost:8000/login/
HTTP_STATUS_LOGIN:302

# 2. Subida y análisis de un documento real
$ curl -s -b cookies.txt -X POST \
  -F "archivo=@media/documentos/ ... /factura.pdf" \
  -w "\nHTTP_STATUS:%{http_code}\n" \
  http://localhost:8000/api/v1/analizar-documento/
```

Respuesta real (201 Created), pipeline completo ejecutado dentro del contenedor y persistido en PostgreSQL, incluyendo la llamada real a Gemini:

```
{
  "documento_id": 1,
  "estado": "analizado",
  "es_documento_tributario": true,
  "es_deducible": true,
  "confianza_clasificacion": 1.0,
  "tipo_documento_detectado": "Factura",
  "entidades": {
    "empresa": "UNIVERSIDAD CAPITAN GENERAL GERARDO BARRIOS",
    "fecha_documento": "29/07/2026",
    "numero_documento": "DTE-01-M001P001-000000000060602"
  },
  "montos": { "subtotal": 143.75, "iva": null, "otros_cargos": null, "total": 143.75 },
  "resumen_ia": "Factura electrónica emitida por la UNIVERSIDAD CAPITAN GENERAL GERARDO BARRIOS ... por concepto de pago de cuota educativa. El monto total de la operación es de $143.75.",
  "recomendacion_ia": "Se recomienda verificar que el monto total declarado por educación no exceda el límite anual de $15,000 para hacer efectiva la deducción ...",
  "justificacion_deducible": "El gasto corresponde a educación, que es deducible hasta $15,000 anuales según la legislación fiscal salvadoreña."
}
HTTP_STATUS:201
```

(Campos con datos personales del documento de prueba —cliente, NIT, dirección— se omitieron de este resumen por privacidad; el esquema completo de respuesta está documentado en `docs/api.md` .)

Esta prueba confirma que, dentro del contenedor, funcionan de punta a punta: extracción de texto, spaCy (entidades), la llamada real a la API de Gemini (clasificación + resumen + recomendación), y la escritura a la base de datos — no es un mock.

9. Errores, intentos, bloqueos y correcciones realizadas

9.1 Bloqueo: Docker Desktop no arrancaba ("Virtualization support not detected")

Síntoma: al abrir Docker Desktop, pantalla de error "Virtualization support not detected", con el motor ("Engine") detenido.

Diagnóstico:

```
> systeminfo | Select-String "Hyper-V" -Context 0,8
Requisitos Hyper-V:  Extensiones de modo de monitor de VM: Sí
                     Se habilitó la virtualización en el firmware: Sí
                     ...
```

La virtualización Sí estaba habilitada a nivel de firmware/BIOS (equipo físico ASUS Vivobook, no una VM). Al consultar las características de Windows relacionadas:

```
> Get-WindowsOptionalFeature -Online -FeatureName VirtualMachinePlatform
Get-WindowsOptionalFeature : La operación requiere elevación.

> wsl.exe --version
(imprime el texto de ayuda en vez de un número de versión → binario desactualizado, WSL2 no
instalado realmente)
```

Explicación técnica: las características de Windows "Windows Subsystem for Linux" y "Virtual Machine Platform" (requeridas por el backend WSL2 de Docker Desktop) no estaban habilitadas, aunque el hardware sí soportaba virtualización.

Ruta de corrección aplicada:

```
# PowerShell como Administrador
dism.exe /online /enable-feature /featurename:Microsoft-Windows-Subsystem-Linux /all
/norestart
dism.exe /online /enable-feature /featurename:VirtualMachinePlatform /all /norestart
# Reinicio de Windows
# Tras reiniciar (PowerShell normal):
wsl --update
wsl --set-default-version 2
```

Después de esto, Docker Desktop inició correctamente (`docker ps` respondió sin error).

9.2 Bug real: `InvalidStorageError` al subir un documento dentro del contenedor

Síntoma: con el contenedor corriendo (`/health` en `200 OK`), el endpoint principal devolvía `500 Internal Server Error` (página genérica, ya que `DEBUG=False`).

Diagnóstico: se levantó una instancia temporal del mismo contenedor con `DEBUG=True` (`docker compose run --rm -e DEBUG=True -p 8001:8000 web ...`) para ver el traceback completo:

```
Exception Type: InvalidStorageError at /api/v1/analizar-documento/  
Exception Value: Could not find config for 'default' in settings.STORAGES.
```

Causa raíz: al agregar whitenoise se definió `STORAGES` con solo la clave `"staticfiles"`. Desde Django 4.2, `STORAGES` reemplaza por completo la config por defecto — al faltar la clave `"default"` (usada por `FileField` / `ImageField`, incluida la subida de documentos), cualquier subida de archivo rompía. No se manifestaba en desarrollo local porque ahí `STORAGES` no estaba definido.

Corrección aplicada (`tributia_project/settings.py`):

```
STORAGES = {  
    "default": {  
        "BACKEND": "django.core.files.storage.FileSystemStorage",  
    },  
    "staticfiles": {  
        "BACKEND": "whitenoise.storage.CompressedManifestStaticFilesStorage",  
    },  
}
```

Verificación: se reconstruyó la imagen (`docker compose up -d --build`) y se repitió la prueba del endpoint principal (sección 8), obteniendo `201 Created` con el pipeline de IA completo funcionando.

9.3 Nota sobre observabilidad

Mientras se investigaba el error 500, se confirmó que con `DEBUG=False` y sin `LOGGING` explícito en `settings.py`, los `logger.exception( ... )` ya existentes en `core/api_views.py` no aparecían en `docker compose logs` (el logger raíz de Django solo envía al handler `console` cuando `DEBUG=True`). Confirma el riesgo ya identificado en `riesgos-tecnicos.md` y refuerza la prioridad de logging estructurado para la Semana 5.

10. Plan de infraestructura mínima

Componente	Propuesta	Justificación
Cómputo	1 contenedor Docker (app + gunicorn), 1 vCPU / 1–2 GB RAM	El pipeline de OCR/spaCy es el punto más pesado en CPU/RAM; con carga baja (proyecto académico) alcanza con un worker pequeño
Base de datos	PostgreSQL — ya migrado dentro del contenedor (servicio <code>db</code>); para producción, apuntar <code>POSTGRES_HOST</code> /credenciales a un Postgres administrado en vez del contenedor local	SQLite no soportaba escritura concurrente real; riesgo ya mitigado, falta solo apuntar a una instancia administrada
Almacenamiento de archivos	Volumen persistente para <code>MEDIA_ROOT</code> , o bucket S3-compatible	Los documentos deben sobrevivir a reinicios/redeploys del contenedor
Cola de tareas (futuro)	Redis + Celery/RQ	El pipeline de análisis es síncrono y bloqueante; mover a cola evita timeouts
Servidor de aplicación	gunicorn detrás de un proxy inverso (nginx / proxy del PaaS)	gunicorn no debe exponerse directo a internet en producción
Archivos estáticos	whitenoise (ya integrado)	Evita depender de un servidor web adicional en un despliegue de bajo tráfico
Variables/secrets	<code>.env</code> fuera de control de versiones, o gestor de secrets del proveedor	Ninguna clave real debe vivir en el repo
Backups	Backup automático diario de la base de datos	Actualmente SQLite no tiene backup

11. Estimación de costos iniciales (supuestos)

Supuestos: tráfico bajo (proyecto académico, decenas de usuarios concurrentes como máximo), sin SLA de alta disponibilidad, un solo entorno.

Ítem	Opción de bajo costo	Estimación mensual (USD)	Supuesto
Hosting del contenedor	Free/hobby de un PaaS (Render, Railway)	\$0 – \$7	Free tier con "sleep" tras inactividad, o primer tier pago para disponibilidad continua
Base de datos PostgreSQL	Free/starter del mismo proveedor	\$0 – \$5	Free tier con límite de almacenamiento (256MB–1GB)
Almacenamiento de archivos	Incluido en el plan, o bucket free tier (Cloudflare R2: 10GB)	\$0	Documentos son PDFs/imágenes de pocos MB
Google Gemini API	Tier gratuito de Google AI Studio	\$0	Limitado por cuota diaria de requests
Dominio (opcional)	Subdominio gratuito del proveedor	\$0	Dominio propio rondaría \$10–15/año
Total estimado (piloto)		\$0 – \$12/mes	Válido dentro de los free tiers

El mayor riesgo de costo no es la infraestructura sino el consumo de la API de Gemini si el proyecto escalara más allá del tier gratuito.

12. Riesgos técnicos pendientes

- 1. **Pipeline síncrono y bloqueante** dentro del request HTTP — un documento grande o Gemini lento puede provocar timeouts del proxy/PaaS. Mitigación planeada: cola de tareas (Celery/RQ).
- 2. ~~SQLite no apto para concurrencia real en producción~~ — **mitigado en esta entrega**: el contenedor corre sobre PostgreSQL. Pendiente: apuntar a una instancia administrada (no el contenedor local) para producción.
- 3. **Sin validación de permisos por usuario** en vistas sensibles — pendiente para Semana 6.
- 4. **Cuota de Gemini limitada** — bajo carga real en un ambiente desplegado, la probabilidad de agotar la cuota diaria sube.
- 5. **Sin backups automáticos** de la base de datos ni de MEDIA_ROOT (el volumen `postgres_data` persiste entre reinicios del contenedor, pero no hay backup fuera de la máquina host).
- 6. **Imagen Docker no integrada al pipeline de CI** — el CI de Semana 3 solo corre `manage.py test` contra SQLite en memoria, no `docker build` ni pruebas contra Postgres. Mejora planeada para Semana 5.
- 7. **Pipeline de análisis aún síncrono y bloqueante** también dentro del contenedor — la migración a Postgres no resuelve esto; sigue pendiente moverlo a cola de tareas (Celery/RQ).

Repositorio actualizado

<https://github.com/DanCastSV/TributIA>

Incluye código fuente, README con instrucciones de ejecución local y con Docker, `Dockerfile` , `.dockerignore` , `.env.example` , pruebas y workflow de CI de Semana 3, y toda la documentación de esta entrega en `docs/despliegue-semana4.md` .